Is Large Language Models' Generated Code Secure?: An Empirical Study

Ikhlaq Ahmad - ixa190000
ixa190000@utdallas.edu

Sandeep Jammula - dal212621
dal212621@utdallas.edu

CS 7301.002 - Dr. Bhavani Thuraisingham

The University of Texas at Dallas
Department of Computer Science

December 8, 2025

Abstract

Recent advances in Artificial Intelligence (AI) and computational power have brought about a new revolution of automation in the software industry. Large Language Models (LLMs) have been significantly improved over the years to generate and deploy code. Although the need for manual programming has been significantly reduced, the substandard quality and security aspects of the generated code has opened a new dimension for potential research. Many studies have deduced that LLM generated code often contains logic flaws, security weaknesses, redundancy, and irrelevant context. In this study, we explore the ability of LLMs to generate code from the beginning and maintain security as the requirements change.

We will compare the code from current state-of-the-art models such as OpenAI, Google, Anthropic, Mistral, etc. The prompts will be instructions given to LLMs to design programs ranging from simple to more complex. All generated programs were kept under 100 lines of code (LoC) for feasibility. Security testing will include static analysis using currently available tools, such as SonarQube and manual analysis. We will also review the latest literature extensively to analyze the current state of the art, research into the subject matter, developers' perspective, and the future of software development in the era of AI.

1 Introduction

The rapid advancement of large language models (LLMs) has fundamentally changed the software development. Publicly available tools such as GitHub Copilot, ChatGPT's Codex, and other AI-assisted programming frameworks can now produce functional code snippets, full modules, and even end-to-end applications with minimal human input. While these AI models are paving the way for the next big revolution in the software industry, this evolution of technology has brought a new security risks that have not yet been systematically addressed.

The main reason for security issues in LLM generated code is the data used for training. LLMs learn statistical correlations from large and often unstructured code corpora from the entire internet; they can inadvertently reproduce insecure patterns, outdated libraries, or vulnerable logic. Several recent studies [1, 2, 3, 4, 5, 6] have revealed that AI-generated code often contains security flaws—such as insufficient input validation, insecure cryptographic practices, missing authentication checks, and flawed concurrency handling—despite appearing syntactically correct and logically plausible in most cases.

Given that the future of development will be significantly based on AI tools; therefore, there is a need for a systematic evaluation of the security properties of AI-generated code. Moreover, current static and dynamic analysis techniques offer partial insight, but may not capture the unique patterns introduced by generative models. Similarly, existing secure coding guidelines were written for human programmers and may not fully account for LLM-specific vulnerability modes. This paper provides an empirical analysis of the security risks inherent in AI-generated code and evaluates existing detection and mitigation strategies. By analyzing the strengths and weaknesses of AI-assisted development, we further advance this area of research to better understand the role and current status of these technologies.

2 Related Work

2.1 Software Engineering and LLMs

Automatic code generation dates back to the 1960s, when Waldinger and Lee introduced PROW, a program synthesizer that generated LISP programs from predicate-calculus specifications [7]. Significant research over the years has drastically improved AI and it's application. In 2017, attention and the transformer architecture mechanism [1, 2] for large language models forever changed the landscape of LLMs; thus, automatic code generation became a reality. LLMs now enable automatic creation of working code, which can significantly enhance developers' efficiency and speed.

In addition, a diverse range of specialized LLMs agents—such as Codex, Polycoder, CodeGen, CodeBERT, StarCoder, and others—have also been introduced to accelerate the entire software development process [4]. Beyond coding, tasks such as test case generation, vulnerability detection, and documentation are also becoming automated. [1, 5]. However, the quality and security of the generated code remains debatable, challenging traditional programming methods, and prompting the need for newer, more robust approaches to address these challenges.

2.2 State of the AI Generated Code

LLMs are advancing rapidly, and so is their ability to produce syntactically correct working code. LLM Prompts are also evolving as software developers become familiar with these models. For example, first time prompts for accurate code have risen dramatically—from about 25% in June 2021 to about 96% by April 2024 [8]. In addition to that, 97% of developers now use some form of AI, and roughly 30% of modern code is produced by AI. [8].

However, current research [9, 10, 11, 12, 13, 14] shows that the overall quality, architecture, and security of the generated code are subpar. Several other studies [9, 10] have mentioned that the use of LLM generated code introduced problems that significantly increased overall development time, mostly due to the fix of those induced problems. It should not be surprising, as these models are trained on billions of lines of code from GitHub repositories [15] and stack overflow pages [16, 3]. These repositories contain human-written code with less regard to the overall security of the program. Therefore, LLMs have an inherent tendency to mimic the semantic and syntactic structures of human generated code.

Code quality metrics fundamentally include correct syntax, logic, size, reliability, maintainability, and security. However, inherently flawed C/C++ repositories' data cause more vulnerabilities in C/C++ code generation. For example, C++ code produced by LLMs often misses directives, mismanages types, and misinterprets APIs, causing frequent compile-time errors, particularly with STL or third-party libraries. A 10% increase in vulnerabilities was observed in LLM-based C programming [10].

As C/C++ languages were used during the early foundation of computing machinery, software security was not a design priority, mainly because the computing environments of the time were fundamentally different from today's networked and adversarial landscape. Most of the software was developed and executed in trusted isolated settings, such as academic laboratories, research institutions, or corporate mainframes. These environments operated under the assumption that all users were legitimate and cooperative, which meant that there was no perceived need for adversarial threat protection. As a result, developers did not treat security as a core requirement. Therefore, it can be deduced that C/C++ programs will have significant security vulnerabilities compared to other languages.

Moreover, AI-generated code exhibits higher vulnerabilities in network security (+18.8%), file operations (+12.4%), and error handling (+12.4%), while human-written code faces more buffer overflow and race condition issues [12]. 46.8% of AI-generated code had critical or high-severity vulnerabilities, compared to 30.8% in human-written code [12]. Other key issues in LLM-based coding include hallucinations and biases that unintentionally favor service providers such as Google or Amazon [8]. Analysis of 17,014 prompts showed that these models strongly favored certain providers, even without explicit user requests [17]. Statistical analysis of 733 code snippets found that about 27.3% had security weaknesses. In Python (124 of 419), 30% were vulnerable, and in JavaScript (76 of 314), 25% had problems. [7].

Therefore, on average, according to [7], incidence of 3 security weaknesses per code fragment, irrespective of the programming language. The Study by [18] indicated that the LLM generated 84% of the Java code and 90% of the Python code lacking documentation. Furthermore, the same study also noted that 61% of the Java code and 75% of the Python code were without exception handling; and 39% of the Java code and 25% of the Python code saw non-specific error messages.

2.3 State of the research

The widespread use of AI-generated code raises a key security question: Can AI code be trusted for an end-to-end automation of software development? Would it be truly safe and free from vulnerabilities? Manual code review, once a standard for security experts, is now impractical due to the scale and speed of AI-generated code deployment [13]. Today, most bug detection methods use testing frameworks, such as verification test benches or formal verifications, to identify security defects. Other detection methods are based on lexical and syntactic analysis [14].

Traditional tools like Valgrind have also failed as they only detect about 53.3% of vulnerabilities [12]. This is mainly due to the unique patterns of AI-generated code. Even the most advanced traditional security tools, such as GitHub's CodeQL, still produce false positives and negatives, as static analysis struggles to capture the various ways code with equivalent semantics can be implemented [19]. Despite many flaws, LLM-generated code security has begun to receive attention among researchers, with several frameworks being developed to detect and address vulnerabilities [12, 20, 19, 13, 14, 17].

However, recent research in [21] found that 32% of LLM generated web applications contained high-severity security vulnerabilities. Automatic adjustments in the web server, PHP code, and database could be exploited by attackers to launch targeted attacks, thus interrupting the normal functioning of op-

erations. Implementing secure coding standards and guidelines, such as those outlined by organizations such as Open Web Application Security Project (OWASP) [22] can help mitigate common programming errors and vulnerabilities.

One of the recent AI-focused code security frameworks is DeVAIC (Detection of Vulnerabilities in AI-generated Code) [13]. It is a Python static analysis tool that uses regex-based rules to detect vulnerabilities across 35 Common Weakness Enumeration (CWE) categories. It handles incomplete code, making it effective for identifying and classifying security issues in AI-generated snippets without custom rules [13]. Another AI-specific framework, CWEval (Common Weakness Enumeration Evaluation), addresses limitations in current secure code assessment methods. It uses validated, security-focused programming tasks with detailed specifications, test oracles, and secure and insecure reference implementations to evaluate the ability of LLMs to generate secure code [19].

Other problems LLMs face in secure code generation is a lack of complete understanding of the coding context, as it relies on complex mathematical models to generate responses, which may be inapplicable or suboptimal for software development [20, 13]. Real world problems can be far more complicated and often require a unique and efficient solution. Another important behavior in [19] was noted that LLMs can produce non-functional solutions to avoid vulnerabilities, prioritizing security over user intent, which can render code unusable in practice. Research in [17] also indicates that inaccurate documentation hinders understanding of LLMs, while missing or incomplete documentation has little effect, highlighting the importance of precise documentation to avoid misguidedness. All of these behaviors can also result in malicious code production if misleading or inaccurate documentation is provided in the prompt.

On the positive side, as noted in [17], while LLMs generate insecure code, with up to 65% of the outputs containing vulnerabilities, skilled engineers can guide them in producing secure code using clear and precise prompts, and breaking complex prompts into simpler ones further improves the results. LLMs will eventually enable engineers to work on additional and complex problems.

2.4 Developers' Perspective

Code quality is a multidimensional measure of how well software meets both functional and non-functional requirements [23]. AI-pair programming tools like GitHub Copilot, ChatGPT, and Amazon CodeWhisperer use NLP and machine learning to generate or complete code based on developer input and context. AI can save time, reduce errors, and aid in learning, but raises challenges in reliability, explainability, ethics, and human-AI trust [23]. Despite the popularity of AI assisted programming, there is little empirical evidence on its effects on code quality and developer satisfaction. Most studies are theoretical or experimental and lack real-world project data, but AI assisted developers have higher satisfaction, enjoyment, motivation, engagement, and fulfillment than non AI-pair developers [23].

According to [23], their research finds that: (1) AI-pair programming positively impacts code quality and developer satisfaction by saving time, reducing errors, enhancing skills, exploring possibilities, providing feedback, and improving quality. (2) It also presents challenges with reliability, explainability, trust, communication, autonomy, and motivation. Another study [24] analyzed subreddit posts up to 9/2023, identifying 68 posts and 122 comments on AI development assistants. Copilot generated vulnerable code in 40% of security-critical tasks, and the use of AI often led to less secure code. These findings indicate that AI is widely adopted, mainly as a supervised assistant [24].

Novice developers make about 16 errors per 100 lines of code, while experienced developers halve this rate, yet errors remain a major risk. LLMs speed up coding but can still produce buggy or insecure

code, creating application failures and mistrust. Novices also struggle with procedural programming, highlighting the need for targeted training [21]. Another study [25] surveyed 105 developers of varying levels of experience to understand their perceptions of AI-generated code security and their suggestions. Their survey ran from May 19 to July 13, 2024 [25]. Their research showed that the participants were 41% junior, 28% med-level, and 31% senior, about half of them being non-professional developers (students, hobbyists, freelancers). Twenty percent reported improved secure coding proficiency with AI, 24% reported a decline, and 56% saw no change. Sixty-five percent of their participants viewed over-reliance on AI without human oversight as the greatest risk. Other key concerns included recurring vulnerabilities (59%), new vulnerabilities (52%), privacy and ethical issues (48%), integration with code/infrastructure (29%) and integration with existing security tools (21%) [25].

2.5 Future of Software Engineering

The future of software development will be highly dependent on AI. Newer and more sophisticated techniques will automate the entire process. However, the accuracy and security of such tools will also evolve; thus presenting new challenges and opportunities for improvement. One such tool, called Automated Program Repair (APR) [26] shows promise for micro-scale code generation, but trusting automatically generated code still remains a key issue. With LLM-based code generation, concerns about correctness and the point at which such code can be safely integrated into a codebase are even more critical [26]. Template-guided repair (e.g., PAR) reduces the search space. However, test overfitting remains a challenge, as patches may pass tests but fail generally; constraint-based approaches use symbolic analysis to generalize beyond test cases [26].

Developers often overestimate tools like GitHub Copilot, skipping code verification and introducing vulnerabilities. Modern tools struggle with code optimization and context, causing errors. Hyper Assistants aim to provide advanced bug detection and fixation for complex software systems [27]. BMC [28], a counterexample-guided repair framework, uses advanced engines for various industrial languages. It models programs as state transition systems from the control-flow graph (CFG), converting SSA statements into SMT formulas solved by SAT/SMT solvers. In the CFG, nodes are operations, and edges represent possible control-flow changes [28].

3 Methodology

We generate 10 real-world inspired coding tasks—five in Java and five in Python—covering scenarios such as peer-to-peer file transfers, REST APIs with local endpoints, SHA-256 file hashing, one-to-many client-server communication, simple chatrooms, and Authorization-Granting Code. Each task was presented as a single-line, zero-shot prompt to evaluate whether current LLMs inherently address security considerations, without any explicit guidance. This setup allowed us to assess the baseline awareness of security and determine whether prompt refinement might be needed to improve security outcomes. Five language models—ChatGPT, DeepSeek, Gemini, Claude, and Mistral were used on free-tier accounts, with trial-limited premium versions used to generate programs. Each prompt was applied uniformly across all models, resulting in a total of 50 programs (5 programs × 2 languages × 5 models). Java programs were compiled to verify the correctness and runtime functionality, while Python programs were executed to ensure that they performed the intended tasks.

Automated security analysis was conducted using SonarQube Cloud and Semgrep for all programs that did not require specialized frameworks. The programs dependent on frameworks were evaluated

using manual analysis, which we deemed suitable for these environments. In addition, we performed a thorough manual inspection of all code files to identify potential vulnerabilities and match them with the corresponding CWE codes. This combined approach allowed a comprehensive assessment of both functional correctness and the security posture of LLM-generated programs.

List of Generated Progams:

- Authorization Grant Simulation
- Peer-to-Peer File Transfer over the Sockets
- SHA 256-bit File Hashing
- Single Server and Multiple Client Chat Room
- REST API using Spring Boot using Local End Points

All coding files, results, and screenshots of sonarQube and Sempgrep are available at: GitHub

4 Results

Based on manual analysis of all the files, we found these CWE-based vulnerabilties:

- **CWE-327: Use of Broken or Risky Cryptographic Algorithm**
- **CWE-330: Use of Insufficiently Random Values**
- **CWE-259: Use of Hard-Coded Password**
- **CWE-200: Exposure of Sensitive Information**
- **CWE-552: Filesystem Exposure**
- **CWE-918: Server-Side Request Forgery (SSRF)**
- **CWE-287: Improper Authentication**
- **CWE-732: Incorrect Permission Assignment for Critical Resource**
- **CWE-399: Resource Management Errors**

Note: Only one screenshot of SonarQube and Semgrep is added to save space.

The security analysis revealed significant variability between models and tasks. Using SonarQube Cloud and Semgrep, we identified multiple common vulnerabilities, including weak or missing input validation, insecure hashing mechanisms (e.g., improper SHA-256 usage), and inadequate authentication or authorization handling. Framework-dependent programs required separate analysis, where similar security gaps were detected. Manual inspection confirmed these findings and allowed us to map each vulnerability to the corresponding CWE code.

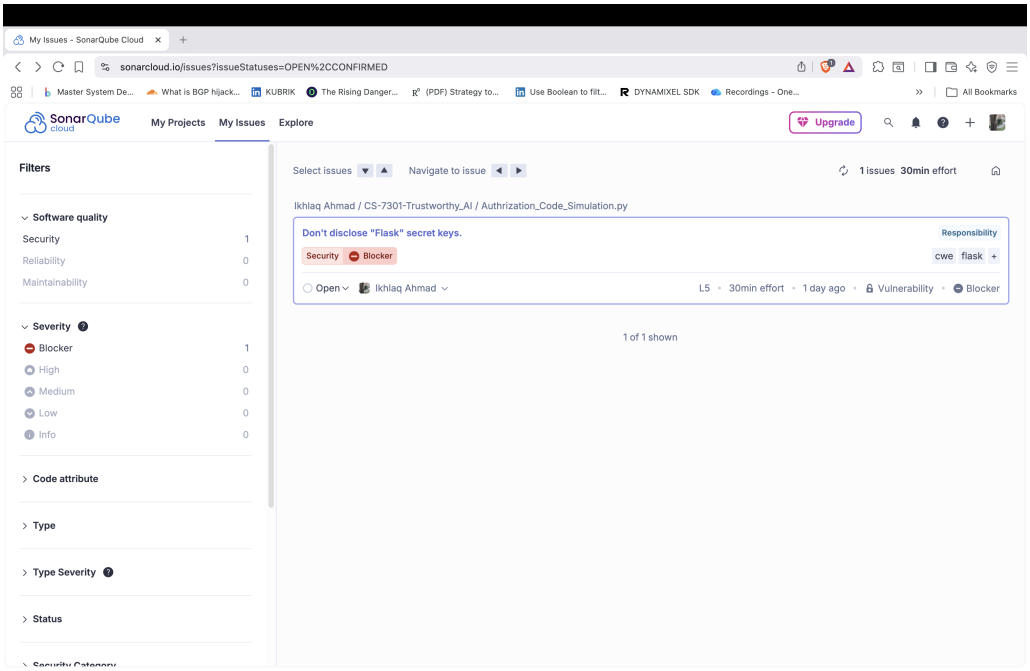

Figure 1: SonarQube analysis highlighting common CWE issues in a Python program.

```
DeepSeek/Java/Simple-Chatroom/ChatServer.java
❯❯ java.lang.security.audit.crypto.unencrypted-socket.unencrypted-socket
      Detected use of a Java socket that is not encrypted. As a result, the traffic could be read by an
      attacker intercepting the network traffic. Use an SSLSocket created by 'SSLSocketFactory' or
      'SSLServerSocketFactory' instead.
      Details: https://sg.run/W8zA

       12┆ try (ServerSocket serverSocket = new ServerSocket(PORT)) {

❯❯ java.lang.security.audit.active-debug-code-printstacktrace.active-debug-code-printstacktrace
      Possible active debug code detected. Deploying an application with debug code can create unintended
      entry points or expose sensitive information.
      Details: https://sg.run/4K8z

       65┆ e.printStackTrace();

DeepSeek/Java/Simple-Chatroom/ConsoleChatClient.java
❯❯ java.lang.security.audit.crypto.unencrypted-socket.unencrypted-socket
      Detected use of a Java socket that is not encrypted. As a result, the traffic could be read by an
      attacker intercepting the network traffic. Use an SSLSocket created by 'SSLSocketFactory' or
      'SSLServerSocketFactory' instead.
      Details: https://sg.run/W8zA

       22┆ socket = new Socket(serverAddress, 12345);

❯❯ java.lang.security.audit.active-debug-code-printstacktrace.active-debug-code-printstacktrace
      Possible active debug code detected. Deploying an application with debug code can create unintended
      entry points or expose sensitive information.
      Details: https://sg.run/4K8z

       66┆ e.printStackTrace();
```

Figure 2: Semgrep analysis highlighting common CWE issues in a Java program.

Comparison of our models revealed that DeepSeek and Mistral resulted in the lengthiest code for the same prompt; DeepSeek generated the code with the most security problems. It also generated irrelevant and unnecessary files and frameworks which could be a simple program. For example, a peer-to-peer file sharing program did not need any framework in any of the models except DeepSeek. ChatGPT generated a java file for SHA 256 bit encryption that did not compile. Gemini generated code with more issues in Java and less issues in Python. By far, the most common issues were the lack of input neutralization and the lack of exception handling.

Quantitatively, automated analysis detected the majority vulnerabilities, but manual inspection was essential to capture subtle security issues and framework-specific concerns. Overall, this demonstrates that while LLMs are capable of producing functional code quickly, significant human oversight is necessary to ensure security compliance, especially for real-world applications. The results suggest that, while LLMs capture general programming patterns and semantics effectively, they do not inherently incorporate secure coding practices without explicit prompting.

5 Discussion and Conclusion

The LLM-generated code should be treated as non-trusted by default. Our study highlights several key insights into the security behavior of LLM-generated code in real-world inspired programming tasks. First, we observed that while current LLMs can generate functional code in both Java and Python, their outputs often lack inherent security considerations. Many programs were executed correctly, but contained vulnerabilities that could lead to data leaks, improper authentication, or weak cryptography practices. This indicates that zero-shot prompts alone are insufficient for producing secure code, emphasizing the need for prompt refinement or post-generation security checks.

The comparison between models revealed variability in security awareness. Some models, particularly those with more advanced training, generated fewer vulnerabilities, but none consistently produced fully secure code. Automated analysis tools, such as SonarQube Cloud and Semgrep, proved effective in identifying common security issues, including improper input validation, weak hashing, and insecure communication patterns. However, framework-dependent programs required specialized analysis tools, demonstrating that automated tools have limitations when applied to complex or non-standard coding environments. Manual inspection was therefore crucial for a complete understanding of security weaknesses and for mapping vulnerabilities to CWE codes.

Our findings suggest several practical implications. First, developers relying on LLM-generated code should treat it as a draft that requires a thorough security review before deployment. Second, incorporating security-focused prompts or fine-tuning models in secure coding datasets could improve the safety of generated programs. Finally, the combination of automated and manual analysis provides the most comprehensive evaluation, highlighting the importance of human oversight even in the presence of advanced LLMs.

In general, this study underscores that while LLMs are promising in accelerating software development, security remains a critical challenge. Future work should explore strategies for integrating security-aware reasoning into LLMs output, such as reinforced learning from secure code examples or hybrid approaches combining LLMs with automated vulnerability detection.

References

[1] Z. Zheng, K. Ning, Q. Zhong, J. Chen, W. Chen, L. Guo, W. Wang, and Y. Wang, "Towards an understanding of large language models in software engineering tasks," *Empirical Software Engineering*, vol. 30, 12 2024.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[3] V. Majdinasab, M. J. Bishop, S. Rasheed, A. Moradidakhel, A. Tahir, and F. Khomh, "Assessing the security of github copilot's generated code - a targeted replication study," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2024, pp. 435–444.

[4] C. Negri-Ribalta, R. Geraud-Stewart, A. Sergeeva, and G. Lenzini, "A systematic literature review on the impact of ai models on the security of code generation," *Frontiers in Big Data*, vol. Volume 7 - 2024, 2024. [Online]. Available: https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2024.1386720

[5] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA Journal of Automatica Sinica*, vol. 12, no. 2, pp. 317–334, 2025.

[6] M. Bruni, F. Gabrielli, M. Ghafari, and M. Kropp, "Benchmarking prompt engineering techniques for secure code generation with gpt models," in *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*, 2025, pp. 93–103.

[7] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, and J. Chen, "Security weaknesses of copilot-generated code in github projects: An empirical study," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 8, Oct. 2025. [Online]. Available: https://doi.org/10.1145/3716848

[8] J. Spracklen, R. Wijewickrama, A. N. Sakib, A. Maiti, B. Viswanath, and M. Jadliwala, "We have a package for you! a comprehensive analysis of package hallucinations by code generating llms," in *Proceedings of the 34th USENIX Conference on Security Symposium*, ser. SEC '25. USA: USENIX Association, 2025.

[9] N. Tihanyi, T. Bisztray, M. A. Ferrag, R. Jain, and L. C. Cordeiro, "How secure is ai-generated code: a large-scale comparison of large language models," *Empirical Softw. Engg.*, vol. 30, no. 2, Dec. 2024. [Online]. Available: https://doi.org/10.1007/s10664-024-10590-1

[10] M. Kharma, S. Choi, M. .Alkhanafseh, and D. Mohaisen, "Security and quality in llm-generated code: A multi-language, multi-model analysis," 02 2025.

[11] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," *Commun. ACM*, vol. 68, no. 2, p. 96–105, Jan. 2025. [Online]. Available: https://doi.org/10.1145/3610721

[12] H. Kim and S. Yoo, "Security analysis of automated code generation: Structural vulnerabilities in ai-generated," *Tehnički glasnik*, vol. 19, pp. 560–574, 09 2025.

[13] D. Cotroneo, R. De Luca, and P. Liguori, "Devaic: A tool for security assessment of ai-generated code," *Information and Software Technology*, vol. 177, p. 107572, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584924001770

[14] B. Ahmad, J. Ah-Kiow, B. Tan, R. Karri, and H. Pearce, "Flag: Finding line anomalies (in rtl code) with generative ai," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 30, no. 6, Oct. 2025. [Online]. Available: https://doi.org/10.1145/3736411

[15] "Github." [Online]. Available: https://github.com/

[16] "Stack overflow." [Online]. Available: https://stackoverflow.com/

[17] H. Jelodar, M. Meymani, and R. Razavi-Far, "Large language models (llms) for source code analysis: applications, models and datasets," 03 2025.

[18] S. Almanasra and K. Suwais, "Analysis of chatgpt-generated codes across multiple programming languages," *IEEE Access*, vol. 13, pp. 23 580–23 596, 2025.

[19] J. Peng, L. Cui, K. Huang, J. Yang, and B. Ray, "Cweval: Outcome-driven evaluation on functionality and security of llm code generation," pp. 33–40, 05 2025.

[20] R. Bouzid and R. Khoury, "Assessing the effectiveness of chatgpt in secure code development: A systematic literature review," *ACM Comput. Surv.*, vol. 57, no. 12, Jul. 2025. [Online]. Available: https://doi.org/10.1145/3744553

[21] T. Mateo Sanguino, "Enhancing security in industrial application development: Case study on self-generating artificial intelligence tools," *Applied Sciences*, vol. 14, p. 3780, 04 2024.

[22] "Owasp top 10," OWASP Foundation, 2021, accessed: 2025-12-08. [Online]. Available: owasp.org

[23] T. Chen, "The impact of ai-pair programmers on code quality and developer satisfaction: Evidence from timi studio," pp. 201–205, 2024.

[24] C. Ludden, C. Burton, and D. Votipka, "An investigation of online developer discussions about generative ai programming assistants," 2024.

[25] A. Kudriavtseva, N. A. Hotak, and O. Gadyatskaya, "My code is less secure with gen ai: Surveying developers' perceptions of the impact of code generation tools on security," in *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing*, ser. SAC '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1637–1646. [Online]. Available: https://doi.org/10.1145/3672608.3707778

[26] M. R. Lyu, B. Ray, A. Roychoudhury, S. H. Tan, and P. Thongtanunam, "Automatic programming: Large language models and beyond," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, May 2025. [Online]. Available: https://doi.org/10.1145/3708519

[27] K. Qiu, N. Puccinelli, M. Ciniselli, and L. Di Grazia, "From today's code to tomorrow's symphony: The ai transformation of developer's routine by 2030," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, May 2025. [Online]. Available: https://doi.org/10.1145/3709353

[28] N. Tihanyi, Y. Charalambous, R. Jain, M. A. Ferrag, and L. C. Cordeiro, " A New Era in Software Security: Towards Self-Healing Software via Large Language Models and Formal Verification ," pp. 136–147, Apr. 2025. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/AST66626.2025.00020